Data understanding and exploration

```
#Load the data using pandas
df = pd.read_csv('/content/drive/MyDrive/Dataset/healthcare_dataset.csv')
```

2. Explore the dataset structure

```
df.columns
```

```
Index(['patient_id', 'age', 'bmi', 'exercise_hours_per_week', 'smoker',
       'blood_pressure', 'diabetes'],
      dtype='object')
```

```
df.head()
```

	patient_id	age	bmi	exercise_hours_per_week	smoker	blood_pressure	diabetes
0	1	31	18.3	1.5	0	119.1	0
1	2	44	26.3	3.1	0	120.5	0
2	3	43	24.2	4.8	0	130.3	0
3	4	32	25.0	6.0	0	119.5	0
4	5	28	22.1	4.2	1	127.4	0

```
# a. shape of the dataset
df.shape
```

```
(6000, 7)
```

```
# b. column types
df.dtypes
```

	0
patient_id	int64
age	int64
bmi	float64
exercise_hours_per_week	float64
smoker	int64
blood_pressure	float64
diabetes	int64

3. Basic statistical analysis

```
df.drop(columns=['patient_id']).describe()
```

	age	bmi	exercise_hours_per_week	smoker	blood_pressure	diabetes
count	6000.000000	6000.000000	6000.000000	6000.000000	6000.000000	6000.000000
mean	51.107333	25.995267	3.012800	0.307000	134.239733	0.090333
std	19.522722	4.960541	1.491802	0.461288	11.234260	0.286683
min	18.000000	15.000000	0.000000	0.000000	94.000000	0.000000
25%	34.000000	22.500000	2.000000	0.000000	126.000000	0.000000
50%	51.000000	26.100000	3.000000	0.000000	134.200000	0.000000
75%	68.000000	29.400000	4.000000	1.000000	142.325000	0.000000
max	84.000000	44.900000	8.500000	1.000000	169.700000	1.000000

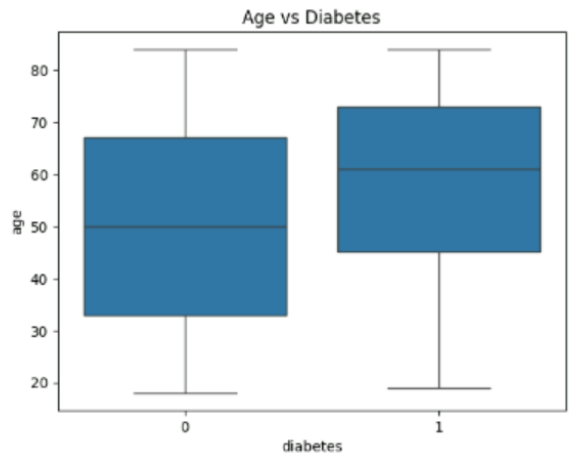
```
df['diabetes'].value_counts()
```

diabetes	count
0	5458
1	542

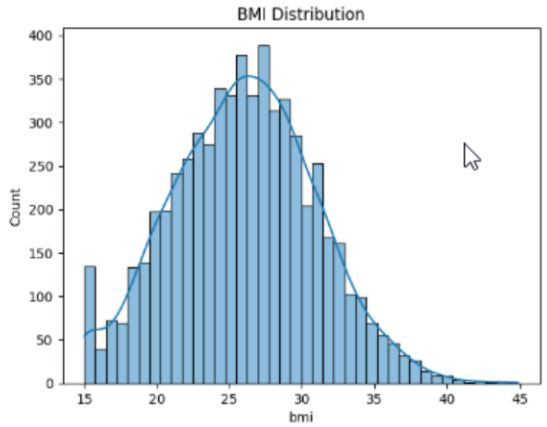
dtype: int64

Visualization

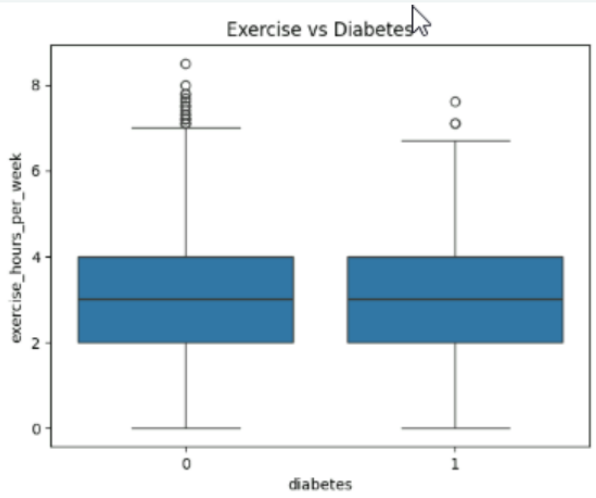
```
sns.boxplot(x='diabetes',y='age', data = df)
plt.title("Age vs Diabetes")
plt.show()
```



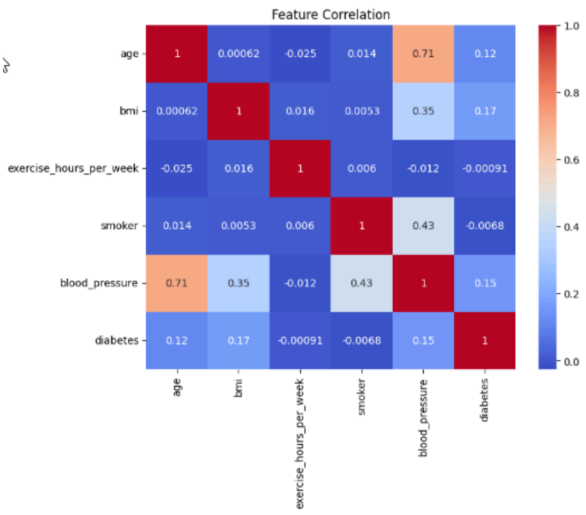

```
sns.histplot(df['bmi'], kde=True)
plt.title("BMI Distribution")
plt.show()
```




```
sns.boxplot(x='diabetes', y='exercise_hours_per_week', data=df)
plt.title("Exercise vs Diabetes")
plt.show()
```



```
plt.figure(figsize=(8,6))

corr = df.drop(columns=['patient_id']).corr()

sns.heatmap(corr, annot=True, cmap='coolwarm')

plt.title("Feature Correlation")
plt.show()
```



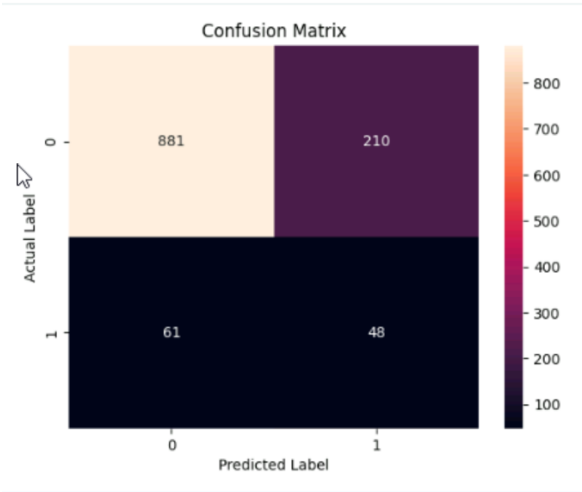
Machine Learning Classification Model

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.7741666666666667
```

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[881 210]
 [ 61  48]]
```



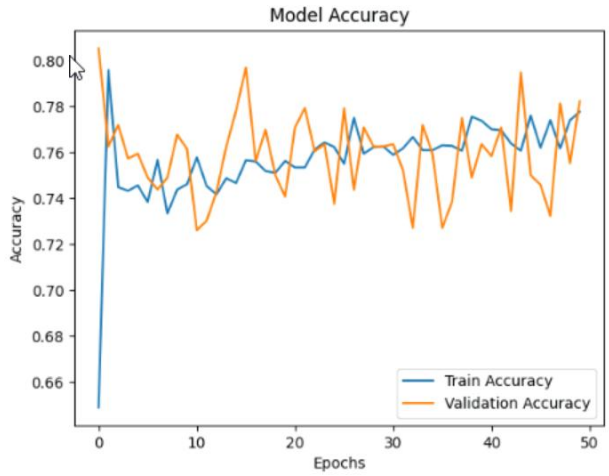
```
# Precision, Recall, F1
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

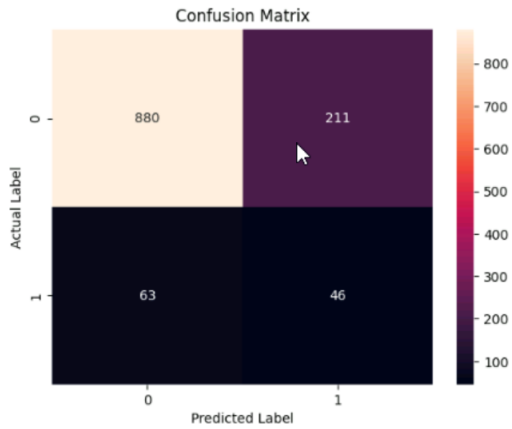
```
Classification Report:
               precision    recall  f1-score   support

           0       0.94      0.81      0.87      1091
           1       0.19      0.44      0.26       109

    accuracy                           0.77      1200
   macro avg       0.56      0.62      0.56      1200
weighted avg       0.87      0.77      0.81      1200
```

Neural Network Model

```
# 8. Evaluate model
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)
```

```
38/38 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7717 - loss: 0.4768
Test Accuracy: 0.7716666460037231
```





```
Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.81      0.87      1091
           1       0.18      0.42      0.25       109

    accuracy                           0.77      1200
   macro avg       0.56      0.61      0.56      1200
weighted avg       0.86      0.77      0.81      1200
```